

Name : Dhiraj Suresh Patil

PRN : 231106024

Roll No : 23

```
import numpy as np
import pandas as pd
```

```
df=pd.read_csv('IMDB Dataset.csv')
```

```
df.head()
```

	review	sentiment
0	One of the other reviewers has mentioned that ...	positive
1	A wonderful little production. The...	positive
2	I thought this was a wonderful way to spend ti...	positive
3	Basically there's a family where a little boy ...	negative
4	Petter Mattei's "Love in the Time of Money" is...	positive

```
#one review
df['review'][1]
```

'A wonderful little production.

The filming technique is very unassuming- very old-time-BBC fashion and gives a comforti ng, and sometimes discomfoting, sense of realism to the entire piece.

The actors are extremely well chosen- Michael She en not only "has got all the polari" but he has all the voices down pat too! You can truly see the seamless editing guided by the re ferences to Williams\' diary entries, not only is it well worth the watching but it is a terrificly written and performed piece. A m asterful production about one of the great master\'s of comedy and his life.

The realism really comes home with the litt

```
df=df.sample(10000)
```

```
df.shape
```

```
(10000, 2)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 10000 entries, 1198 to 31466
Data columns (total 2 columns):
#   Column      Non-Null Count  Dtype
---  ---
0    review      10000 non-null   object
1    sentiment    10000 non-null   object
dtypes: object(2)
memory usage: 234.4+ KB
```

```
df['sentiment'].replace({'positive' : 1,'negative' : 0}, inplace = True)
```

/tmp/ipykernel_4307/2471294948.py:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained as The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting v

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col

```
df['sentiment'].replace({'positive' : 1,'negative' : 0}, inplace = True)
/tmp/ipykernel_4307/2471294948.py:1: FutureWarning: Downcasting behavior in `replace` is deprecated and will be removed in a future v
df['sentiment'].replace({'positive' : 1,'negative' : 0}, inplace = True)
```

```
df.head()
```

	review	sentiment
1198	This is a great film with an amazing cast. Cri...	1
45963	My mother keeps a cassette of this film as a g...	0
24840	To begin with, I have to admit I've never been...	0
31221	John Waters has given us a genuinely enjoyable...	1
35791	For shame, for shame that a fine actor such as...	0

```
import re
clean = re.compile('<.*?>')
re.sub(clean, '',df.iloc[2].review)
```

'To begin with, I have to admit I've never been a big fan of the Dutch movie-genre. Unfortunately watching De Dominee hasn't made me change my mind. It shares some common flaws, like having a plot that's too predictable and linear for my taste. Worse however is that the cast has their dialogues as if they were stage-actors in a play. Unfortunately this is common too in many Dutch movies and, at least to me, makes it impossible to feel any kind of involvement with the main characters. The actor that plays Adri (I'm sorry, I forgot his name at the moment) is at least delivering a decent performance, and is one of the reasons I don't rate this movie even worse.

```
#Function to clean html tags
def clean_html(text):
    clean = re.compile('<.*?>')
    return re.sub(clean, '',text)
```

```
df['review'] = df['review'].apply(clean_html)
```

```
#converting everything to lowercase
def convert_lower(text):
    return text.lower()
```

```
df['review'] = df['review'].apply(convert_lower)
```

```
#Function to remove special characters
def remove_special(text):
    x= ''
    for i in text:
        if i.isalnum():
            x=x+i
        else:
            x = x + ' '
    return x
```

```
remove_special('th%e @ movie is awesome')
```

```
' th e    movie is awesome'
```

```
df['review'] = df['review'].apply(remove_special)
```

```
#Remove stopwords
import nltk
nltk.download('stopwords')
from nltk.corpus import stopwords
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
```

```
def remove_stopwords(text):
    x=[]
    for i in text.split():
        if i not in stopwords.words('english'):
            x.append(i)
    y=x[:]
    x.clear()
    return y
```

```
df['review'] = df['review'].apply(remove_stopwords)
```

```
df
```

	review	sentiment
1198	[great, film, amazing, cast, crispen, glover, ...	1
45963	[mother, keeps, cassette, film, general, threa...	0
24840	[begin, admit, never, big, fan, dutch, movie, ...	0
31221	[john, waters, given, us, genuinely, enjoyable...	1
35791	[shame, shame, fine, actor, joseph, fiennes, w...	0
...	...	...
15440	[sure, people, saw, movie, apparently, follow,...	1
30516	[john, candy, need, say, main, reason, see, fi...	0
43788	[simply, epitome, made, tv, movie, lazy, sunda...	1
6486	[movie, good, westerns, made, anthony, mann, j...	1
31466	[fake, plot, seems, like, generic, adaptation,...	0

10000 rows × 2 columns

```
#Perform Stemming
from nltk.stem.porter import PorterStemmer
ps = PorterStemmer()
```

```
y=[]
def stem_words(text):
    for i in text:
        y.append(ps.stem(i))
    z=y[:]
    y.clear()
    return z
```

```
df['review'] = df['review'].apply(stem_words)
```

```
#Join back
def join_back(list_input):
    return " ".join(list_input)
```

```
df['review'] = df['review'].apply(join_back)
```

```
df
```

	review	sentiment
1198	great film amaz cast crispen glover freakiest ...	1
45963	mother keep cassett film gener threat film lov...	0
24840	begin admit never big fan dutch movi genr unfo...	0
31221	john water given us genuin enjoy film certainl...	1
35791	shame shame fine actor joseph fienn would allo...	0
...	...	...
15440	sure peopl saw movi appar follow complic plot ...	1
30516	john candi need say main reason see film peopl...	0
43788	simpli epitom made tv movi lazi sunday afterno...	1
6486	movi good western made anthoni mann jame stewa...	1
31466	fake plot seem like gener adapt averag blaxplo...	0

10000 rows × 2 columns

```
df['review']
```

	review
1198	great film amaz cast crispen glover freakiest ...
45963	mother keep cassett film gener threat film lov...
24840	begin admit never big fan dutch movi genr unfo...
31221	john water given us genuin enjoy film certainl...
35791	shame shame fine actor joseph fienn would allo...
...	...
15440	sure peopl saw movi appar follow complic plot ...
30516	john candi need say main reason see film peopl...
43788	simpli epitom made tv movi lazi sunday afterno...
6486	movi good western made anthoni mann jame stewa...
31466	fake plot seem like gener adapt averag blaxplo...

10000 rows × 1 columns

```
dtype: object
```

```
X=df.iloc[:,0:1].values
```

```
X.shape
```

```
(10000, 1)
```

```
from sklearn.feature_extraction.text import CountVectorizer
cv = CountVectorizer()
```

```
X=cv.fit_transform(df['review']).toarray()
```

```
X.shape
```

```
(10000, 35980)
```

```
X[0]
```

```
array([0, 0, 0, ..., 0, 0, 0])
```

```
X[0].mean()
```

```
np.float64(0.0006670372429127293)
```

```
y = df.iloc[:, -1].values
```

```
y
```

```
array([1, 0, 0, ..., 1, 1, 0])
```

```
y.shape
```

```
(10000,)
```

```
#Splitting data
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=0.2)
```

```
X_train.shape
```

```
(8000, 35980)
```

```
from sklearn.naive_bayes import GaussianNB, MultinomialNB, BernoulliNB
```

```
clf1 = GaussianNB()
clf2 = MultinomialNB()
clf3 = BernoulliNB()
```

```
clf1.fit(X_train, y_train)
clf2.fit(X_train, y_train)
clf3.fit(X_train, y_train)
```

```
▼ BernoulliNB ⓘ ?
BernoulliNB()
```

```
y_pred1 = clf1.predict(X_test)
y_pred2 = clf2.predict(X_test)
y_pred3 = clf3.predict(X_test)
```

```
print("Actual:",y_test)
print("Prediction from Clf1:",y_pred1)
```

```
Actual: [1 0 0 ... 1 0 1]
Prediction from Clf1: [0 0 0 ... 0 0 1]
```

```
from sklearn.metrics import accuracy_score, confusion_matrix, precision_score
```

```
print("Accuracy of Gaussian:",accuracy_score(y_test, y_pred1))
print("Accuracy of Miltinomial:",accuracy_score(y_test, y_pred2))
print("Accuracy of Bernaulli:",accuracy_score(y_test, y_pred3))
```

```
Accuracy of Gaussian: 0.631
Accuracy of Miltinomial: 0.812
Accuracy of Bernaulli: 0.7835
```

```
##Test on new reviews
new_reviews = [
    "excellent movie. worth to watch it for 3 hours"
]
```

```
new_vec = cv.transform(new_reviews)
predictions = clf2.predict(new_vec)
for msg, pred in zip(new_reviews, predictions):
    print(msg, "=>", predictions)
```

```
excellent movie. worth to watch it for 3 hours => [1]
```